

Week 2: Running the Programs in VS Code

Dr M AOUDE

Guide to the week 2 programs

About this guide. It explains how to run the week 2 programs in **Visual Studio Code**, and gives a short description of each program in the folder `code/week02/slides`. Each program file also starts with a comment that says what it teaches and what output to expect: run it, then compare your result with that comment.

1 What you need

The same setup as week 1: **Flutter** (it includes the Dart SDK, so the `dart` command works), and **Visual Studio Code** with the **Flutter** extension. If something is missing, follow the install guide `001_install_vscode` and the week 1 guide *Running the programs in VS Code*. All week 2 programs are plain Dart programs: no Flutter project is needed.

2 Get the week 2 code and open it

If you already copied the repository for week 1, update it: open a terminal in the repository folder and type

```
git pull
```

Otherwise download it with `git clone https://github.com/maoude/mobile_pro_2026.git`. In VS Code choose **File** → **Open Folder...** and open `code/week02`. The programs are in the folder `slides`.

3 Run a program

With the Run button

Open the file from the Explorer, then click **Run** in the small **Run | Debug** line above `void main()` (or press F5; `Ctrl+F5` runs without the debugger). The output appears in the **Debug Console** at the bottom.

With the terminal

Open a terminal with **Terminal** → **New Terminal** (`Ctrl+``), go to the folder and use `dart run`:

```
cd code/week02/slides
dart run 02_factorial_function.dart
```

The program that asks you to type

01_error_handling.dart reads what you type. The Debug Console cannot receive keyboard input, so run this one in the **terminal**, type your answers, and press Enter after each one. (If you prefer the Run button, add "console": "terminal" to the launch configuration in .vscode/launch.json.)

4 The programs

Each entry gives the file, what it shows, and the output you should get.

1. Error handling

01_error_handling.dart

Asks for two numbers and prints their sum. If you type something that is not a number, `int.parse` throws a `FormatException`: the program catches it with `try` and `on FormatException`, prints a message, and asks again. A `do-while` loop repeats until both numbers are valid. Run it in the terminal. Output when you first type `abc` by mistake:

```
Enter x: abc
Only numbers are allowed
Enter x: 10
Enter y: 20
Sum is: 30
```

2. Factorial function

02_factorial_function.dart

A function with a parameter and a return value. `factorial(x)` multiplies $1 \times 2 \times \dots \times x$ in a loop, and `main` calls it three times.

```
Factorial of 10 is 3628800
Factorial of 5 is 120
Factorial of 0 is 1
```

3. Optional parameters with a default

03_optional_positional_default.dart

Dart has no function overloading. Instead, a parameter inside `[ ]` is optional, and `= 0` gives it a default value, so one function can be called with one or two arguments.

```
f(10) 20
f(10, 20) 40
```

4. Optional parameters that can be null

04_optional_positional_nullable.dart

An optional parameter with no default value is `null` when omitted, so its type must be `int?`. The function tests for `null`, and a shorter version uses `??`.

```
f(10) result is: 20
f(10, 20) result is: 40
f2(10) result is: 20
f2(10, 20) result is: 40
```

5. Named parameters

05_named_parameters.dart

Parameters inside `{ }` are passed by name, so the order does not matter. They need a default value or the word `required`.

```
f1() result is: 0
f1(x: 10) result is: 10
f1(y: 10) result is: 30
f1(x: 5, y: 10) result is: 35
f1(y: 10, x: 5) result is: 35
Hello, Rami!
Welcome, Rami!
```

6. Lambda functions

06_lambda_functions.dart

A function whose body is one expression can be written with an arrow (`=>`), without `return` or braces.

```
sum(10, 20) result is: 30
Hello Jane
square(7) is: 49
```

7. A function is a value

07_function_type.dart

A function can be stored in a variable. `sayHello` is the function itself; `sayHello()` calls it.

```
Hello world!
Hello world!
HELLO WORLD!
```

8. Anonymous functions

08_anonymous_functions.dart

Functions without a name, written where they are passed to another function. The parameter type `int Function(int)` means “a function that takes an `int` and returns an `int`”.

```
Result: 110
Result: 21
triple(4) is: 12
```

9. Lists, loops and `List.filled`

09_fixed_size_lists.dart

Prints a list of colours with a `for` loop and with `forEach`, then uses `List.filled` to store four grades and prints the passed ones (80.0, 60.0 and 90.0). It also shows that a `List.filled` list has a fixed length: adding an item causes an `UnsupportedError`, which the program catches.

10. Growable lists

10_dynamic_lists.dart

An empty list created with `[]` grows with `add`; `addAll` adds all the items of another list.

```
Even numbers...
6
4
Number of items: 4
[Welcome, to, CSCI410]
```

11. Maps

11_maps.dart

Key/value pairs created from a literal and with the `Map` constructor. A value is read by its key, and a missing key gives `null`.

```
Id: 101
Name: rami
{exam1: 80.0, exam2: 60.0, final: 77.0}
Student with id 303 grade is: 80.0
Student with id 999 grade is: null
Number of students: 4
```

12. Generics and literals

12_generics_and_literals.dart

`<String>[...]` and `<String, String>{...}` state the type of the contents, so Dart rejects the wrong type. The program prints one list and one map of Avengers quotes.

```
List: [Hulk, Captain America]
Map: {Captain America: I can do this all day!, Spider Man: Am I an Avenger?,
      Hulk: Smaaaaaash!}
```

13. Sets**13_sets.dart**

A set keeps only unique items. The program computes the intersection, union and difference of two sets, and shows that `add` returns `false` for an item that is already there.

```
Intersection: {A, B}
Union: {A, B, C, E}
Difference: {C}
Adding 'A' again: false
Adding 'Z': true
```

14. Passing and returning lists**14_lists_and_functions.dart**

One function takes a list and returns the average as a `double`; another returns a new list with only the passed grades. The last line does the same with `where`.

```
Average is: 60.0
Passed grades...
60.0
70.0
Same with where(): [60.0, 70.0]
```

15. The math library**15_math_package.dart**

`import 'dart:math';` adds `pow`, `sqrt`, `max`, `min` and `pi`.

```
2.0 to the power 25.0 is 33554432.0
Square root of 25.0 is 5.0
max(3, 7) = 7
min(3, 7) = 3
pi = 3.141592653589793
```

5 Quick troubleshooting

Problem	What to do
<code>dart</code> is not recognised	Close all terminals and restart VS Code so it sees the new PATH; check with <code>flutter doctor</code> .
No Run Debug line above <code>main</code>	Install the Flutter (Dart) extension, and open the folder that contains the file.
A program waits and nothing happens	It is waiting for typed input: run it in the terminal, not the Debug Console.
The week 2 folder is missing	You have an old copy: run <code>git pull</code> in the repository folder.

This document is maintained by Dr. Mohamad AOUBE
Last updated: 2026-09-21